

(in that same case, a complex-conjugate pair). Equations (5.6.13) – (5.6.16) are arranged both to minimize roundoff error and also (as pointed out by A.J. Glassman) to ensure that no choice of branch for the complex cube root can result in the spurious loss of a distinct root.

If you need to solve many cubic equations with only slightly different coefficients, it is more efficient to use Newton's method (§9.4).

CITED REFERENCES AND FURTHER READING:

- Weast, R.C. (ed.) 1967, *Handbook of Tables for Mathematics*, 3rd ed. (Cleveland: The Chemical Rubber Co.), pp. 130–133.
- Pachner, J. 1983, *Handbook of Numerical Analysis Applications* (New York: McGraw-Hill), §6.1.
- McKelvey, J.P. 1984, "Simple Transcendental Expressions for the Roots of Cubic Equations," *American Journal of Physics*, vol. 52, pp. 269–270; see also vol. 53, p. 775, and vol. 55, pp. 374–375.

5.7 Numerical Derivatives

Imagine that you have a procedure that computes a function $f(x)$, and now you want to compute its derivative $f'(x)$. Easy, right? The definition of the derivative, the limit as $h \rightarrow 0$ of

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad (5.7.1)$$

practically suggests the program: Pick a small value h ; evaluate $f(x+h)$; you probably have $f(x)$ already evaluated, but if not, do it too; finally, apply equation (5.7.1). What more needs to be said?

Quite a lot, actually. Applied uncritically, the above procedure is almost guaranteed to produce inaccurate results. Applied properly, it can be the right way to compute a derivative only when the function f is *fiercely* expensive to compute; when you already have invested in computing $f(x)$; and when, therefore, you want to get the derivative in no more than a single additional function evaluation. In such a situation, the remaining issue is to choose h properly, an issue we now discuss.

There are two sources of error in equation (5.7.1), truncation error and roundoff error. The truncation error comes from higher terms in the Taylor series expansion,

$$f(x+h) = f(x) + hf'(x) + \frac{1}{2}h^2 f''(x) + \frac{1}{6}h^3 f'''(x) + \cdots \quad (5.7.2)$$

whence

$$\frac{f(x+h) - f(x)}{h} = f' + \frac{1}{2}hf'' + \cdots \quad (5.7.3)$$

The roundoff error has various contributions. First there is roundoff error in h : Suppose, by way of an example, that you are at a point $x = 10.3$ and you blindly choose $h = 0.0001$. Neither $x = 10.3$ nor $x + h = 10.30001$ is a number with an exact representation in binary; each is therefore represented with some fractional error characteristic of the machine's floating-point format, ϵ_m , whose value in single precision may be $\sim 10^{-7}$. The error in the *effective* value of h , namely the difference between $x+h$ and x as represented in the machine, is therefore on the order of $\epsilon_m x$,

which implies a fractional error in h of order $\sim \epsilon_m x / h \sim 10^{-2}$! By equation (5.7.1), this immediately implies at least the same large fractional error in the derivative.

We arrive at Lesson 1: Always choose h so that $x + h$ and x differ by an exactly representable number. This can usually be accomplished by the program steps

$$\begin{aligned} \text{temp} &= x + h \\ h &= \text{temp} - x \end{aligned} \quad (5.7.4)$$

Some optimizing compilers, and some computers whose floating-point chips have higher internal accuracy than is stored externally, can foil this trick; if so, it is usually enough to declare `temp` as `volatile`, or else to call a dummy function `donothing(temp)` between the two equations (5.7.4). This forces `temp` into and out of addressable memory.

With h an “exact” number, the roundoff error in equation (5.7.1) is approximately $e_r \sim \epsilon_f |f(x)/h|$. Here ϵ_f is the fractional accuracy with which f is computed; for a simple function this may be comparable to the machine accuracy, $\epsilon_f \approx \epsilon_m$, but for a complicated calculation with additional sources of inaccuracy it may be larger. The truncation error in equation (5.7.3) is on the order of $e_t \sim |hf''(x)|$. Varying h to minimize the sum $e_r + e_t$ gives the optimal choice of h ,

$$h \sim \sqrt{\frac{\epsilon_f f}{f''}} \approx \sqrt{\epsilon_f} x_c \quad (5.7.5)$$

where $x_c \equiv (f/f'')^{1/2}$ is the “curvature scale” of the function f or the “characteristic scale” over which it changes. In the absence of any other information, one often assumes $x_c = x$ (except near $x = 0$, where some other estimate of the typical x scale should be used).

With the choice of equation (5.7.5), the fractional accuracy of the computed derivative is

$$(e_r + e_t)/|f'| \sim \sqrt{\epsilon_f} (f f'' / f'^2)^{1/2} \sim \sqrt{\epsilon_f} \quad (5.7.6)$$

Here the last order-of-magnitude equality assumes that f , f' , and f'' all share the same characteristic length scale, which is usually the case. One sees that the simple finite difference equation (5.7.1) gives *at best* only the square root of the machine accuracy ϵ_m .

If you can afford two function evaluations for each derivative calculation, then it is significantly better to use the symmetrized form

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad (5.7.7)$$

In this case, by equation (5.7.2), the truncation error is $e_t \sim h^2 f'''$. The roundoff error e_r is about the same as before. The optimal choice of h , by a short calculation analogous to the one above, is now

$$h \sim \left(\frac{\epsilon_f f}{f'''} \right)^{1/3} \sim (\epsilon_f)^{1/3} x_c \quad (5.7.8)$$

and the fractional error is

$$(e_r + e_t)/|f'| \sim (\epsilon_f)^{2/3} f^{2/3} (f''')^{1/3} / f' \sim (\epsilon_f)^{2/3} \quad (5.7.9)$$

which will typically be an order of magnitude (single precision) or two orders of magnitude (double precision) *better* than equation (5.7.6). We have arrived at Lesson 2: Choose h to be *the correct* power of ϵ_f or ϵ_m times a characteristic scale x_c .

You can easily derive the correct powers for other cases [1]. For a function of two dimensions, for example, and the mixed derivative formula

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{[f(x+h, y+h) - f(x+h, y-h)] - [f(x-h, y+h) - f(x-h, y-h)]}{4h^2} \quad (5.7.10)$$

the correct scaling is $h \sim \epsilon_f^{1/4} x_c$.

It is disappointing, certainly, that no simple finite difference formula like equation (5.7.1) or (5.7.7) gives an accuracy comparable to the machine accuracy ϵ_m , or even the lower accuracy to which f is evaluated, ϵ_f . Are there no better methods?

Yes, there are. All, however, involve exploration of the function's behavior over scales comparable to x_c , plus some assumption of smoothness, or analyticity, so that the high-order terms in a Taylor expansion like equation (5.7.2) have some meaning. Such methods also involve multiple evaluations of the function f , so their increased accuracy must be weighed against increased cost.

The general idea of “Richardson’s deferred approach to the limit” is particularly attractive. For numerical integrals, that idea leads to so-called Romberg integration (for review, see §4.3). For derivatives, one seeks to extrapolate, to $h \rightarrow 0$, the result of finite difference calculations with smaller and smaller finite values of h . By the use of Neville’s algorithm (§3.2), one uses each new finite difference calculation to produce both an extrapolation of higher order and also extrapolations of previous, lower, orders but with smaller scales h . Ridders [2] has given a nice implementation of this idea; the following program, `dfridr`, is based on his algorithm, modified by an improved termination criterion. Input to the routine is a function f (called `func`), a position x , and a *largest* stepsize h (more analogous to what we have called x_c above than to what we have called h). Output is the returned value of the derivative and an estimate of its error, `err`.

```
template<class T>
Doub dfridr(T &func, const Doub x, const Doub h, Doub &err)
Returns the derivative of a function func at a point x by Ridders' method of polynomial extrapolation. The value h is input as an estimated initial stepsize; it need not be small, but rather should be an increment in x over which func changes substantially. An estimate of the error in the derivative is returned as err.
{
    const Int ntab=10;           Sets maximum size of tableau.
    const Doub con=1.4, con2=(con*con);   Stepsize decreased by CON at each iteration.
    const Doub big=numeric_limits<Doub>::max();
    const Doub safe=2.0;         Return when error is SAFE worse than the
    Int i,j;                     best so far.
    Doub errt,fac,hh,ans;
    MatDoub a(ntab,ntab);
    if (h == 0.0) throw("h must be nonzero in dfridr.");
    hh=h;
    a[0][0]=(func(x+hh)-func(x-hh))/(2.0*hh);
    err=big;
    for (i=1;i<ntab;i++) {
        Successive columns in the Neville tableau will go to smaller stepsizes and higher orders of extrapolation.
        hh /= con;
        a[0][i]=(func(x+hh)-func(x-hh))/(2.0*hh);   Try new, smaller stepsize.
        fac=con2;
```

`dfridr.h`

```

for (j=1;j<=i;j++) {      Compute extrapolations of various orders, requiring
a[j][i]=(a[j-1][i]*fac-a[j-1][i-1])/(fac-1.0);      no new function eval-
fac=con2*fac;      uations.
errrt=MAX(abs(a[j][i]-a[j-1][i]),abs(a[j][i]-a[j-1][i-1]));
The error strategy is to compare each new extrapolation to one order lower, both
at the present stepsize and the previous one.
if (errrt <= err) {      If error is decreased, save the improved answer.
    err=errrt;
    ans=a[j][i];
}
}
if (abs(a[i][i]-a[i-1][i-1]) >= safe*err) break;
If higher order is worse by a significant factor SAFE, then quit early.
}
return ans;
}

```

In `dfridr`, the number of evaluations of `func` is typically 6 to 12, but is allowed to be as great as $2 \times \text{NTAB}$. As a function of input h , it is typical for the accuracy to get *better* as h is made larger, until a sudden point is reached where nonsensical extrapolation produces an early return with a large error. You should therefore choose a fairly large value for h but monitor the returned value `err`, decreasing h if it is not small. For functions whose characteristic x scale is of order unity, we typically take h to be a few tenths.

Besides Ridders' method, there are other possible techniques. If your function is fairly smooth, and you know that you will want to evaluate its derivative many times at arbitrary points in some interval, then it makes sense to construct a Chebyshev polynomial approximation to the function in that interval, and to evaluate the derivative directly from the resulting Chebyshev coefficients. This method is described in §5.8 – §5.9, following.

Another technique applies when the function consists of data that is tabulated at equally spaced intervals, and perhaps also noisy. One might then want, at each point, to least-squares *fit* a polynomial of some degree M , using an additional number n_L of points to the left and some number n_R of points to the right of each desired x value. The estimated derivative is then the derivative of the resulting fitted polynomial. A very efficient way to do this construction is via Savitzky-Golay smoothing filters, which will be discussed later, in §14.9. There we will give a routine for getting filter coefficients that not only construct the fitting polynomial but, in the accumulation of a single sum of data points times filter coefficients, evaluate it as well. In fact, the routine given, `savgol`, has an argument `ld` that determines which derivative of the fitted polynomial is evaluated. For the first derivative, the appropriate setting is `ld=1`, and the value of the derivative is the accumulated sum divided by the sampling interval h .

CITED REFERENCES AND FURTHER READING:

- Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.), §5.4 – §5.6.[1]
 Ridders, C.J.F. 1982, "Accurate computation of $F'(x)$ and $F'(x)F''(x)$," *Advances in Engineering Software*, vol. 4, no. 2, pp. 75–76.[2]